

Dendê Eventos – Refatorando o Código

Autores: Antônio Gabriel Gomes Falcão do Nascimento
Giovanna Oliveira Araujo
Gustavo Costa Pereira
João Magalhães Mira Neto

Agenda

O objetivo desta apresentação é exibir a aplicação Dendê Eventos em seu estágio refatorado.

1. Contexto

Continuação do desafio do programa de trainee do Dendê Softhouse

2. Funções Refatoradas

Decomposição do fluxo em 15 funções mapeadas para cada User Story.

3. Apresentação da Estrutura de Arquivos Final

Exibição da organização por arquivos

4. Apresentação do Model.kt

Centralização de Data Classes e Enums

5. Operações do Repositório

Gerenciamento de persistência em memória

6. Funções de Componente e Dois Formulários Criado com Elas

Entradas com validação, tratamento de erros e exibição tabular

7. Resultado da Refatoração do Método main

Código simplificado e orquestrado.

8. Projeto de Telas

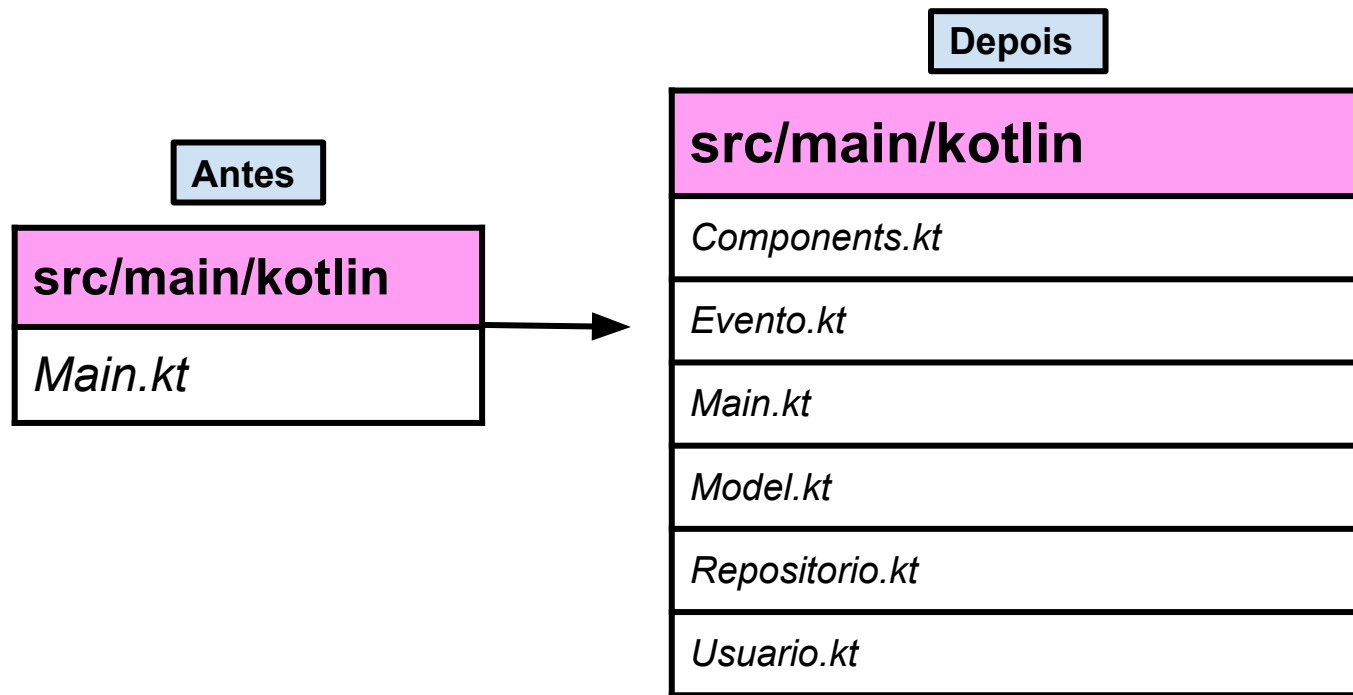
Exibição das telas de Listar ingressos e Login

1. Contexto

Modelo de Recomendação para Dendê-Eventos

Etapas		
1	<i>Aplicação linear na Main</i>	
2	Refatoração	
3	<i>Implementação de Interface</i>	
4	<i>Aplicativo Mobile</i>	

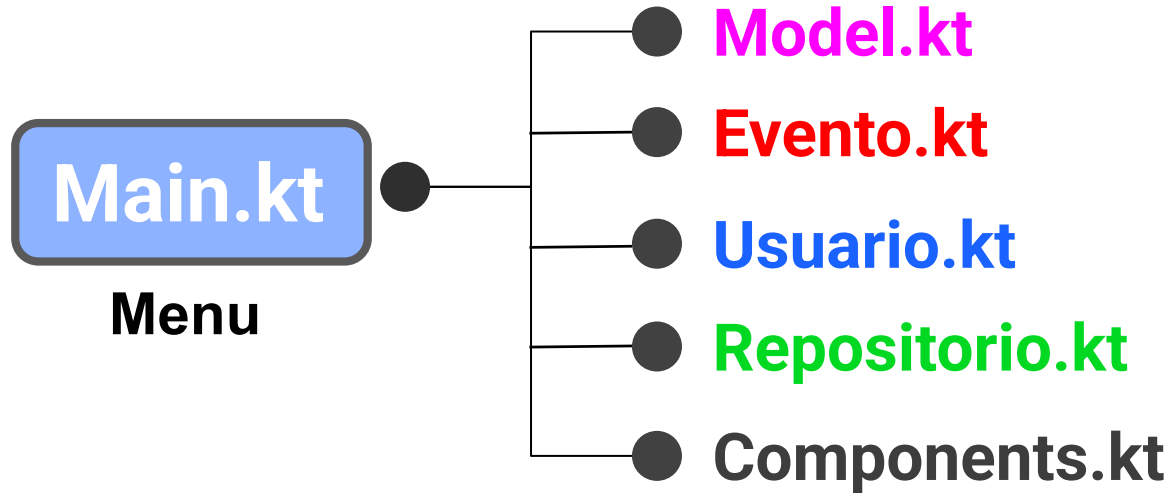
2. Funções Refatoradas



2.1 Funções Refatoradas

src/main/kotlin	
<i>Components.kt</i>	Leitura e tratamento de erro
<i>Evento.kt</i>	Cadastro e alteração de Evento
<i>Main.kt</i>	Menu
<i>Model.kt</i>	Enum class, Data Class
<i>Repositorio.kt</i>	CRUD
<i>Usuario.kt</i>	Cadastro e alteração Usuário

3. Estrutura de Arquivos Final



4. Apresentação do Model.kt

Enum Gênero	
Enum Tipo Evento	
Enum Modalidade	
Data Class User	
Data Class Evento	
Data Class Ingresso	

```
enum class Modalidade {  
    PRESENCIAL, REMOTO, HÍBRIDO  
}
```

```
enum class Gender {  
    MASCULINO,  
    FEMININO  
}
```

```
enum class TipoEvento {  
    SOCIAL, CORPORATIVO, ACADÊMICO, CULTURAL, RELIGIOSOS, ESPORTIVOS,  
    FEIRA, CONGRESSO, OFICINA, CURSO, TREINAMENTO, AULA, SEMINÁRIO,  
    PALESTRA, SHOW, FESTIVAL, EXPOSIÇÃO, RETIRO, CULTO,  
    CELEBRAÇÃO, CAMPEONATO, CORRIDA  
}
```

4.1 Apresentação do Model.kt



```
data class User(  
    var username: String,  
    var birthday: LocalDate = LocalDate.parse("2000-01-01"),  
    val email: String, // ID único para busca  
    var gender: Gender,  
    var password: String,  
    var isOrganizer: Boolean,  
    var cnpj: String = "",  
    var razaoSocial: String = "",  
    var nomeFantasia: String = "",  
    var isActive: Boolean = true  
) {  
    override fun toString(): String {  
        val tipo = if (isOrganizer) "ORG" else "USER"  
        val status = if (isActive) "✓" else "X"  
        return "${status.padEnd(3)} | ${username.padEnd(15)} | ${email.padEnd(25)} | ${tipo.padEnd(5)} | ${gender.name.padEnd(10)}"  
    }  
}
```


4.2 Apresentação do Model.kt



```
data class Evento(  
    var nomeEvento: String,  
    var descricao: String,  
    var dataInicio: LocalDateTime,  
    var dataFim: LocalDateTime,  
    var eventoPrincipal: String,  
    var capacidadeMaxima: Int = 0,  
    var localEvento: String,  
    var precoUnitario: Float = 0.0F,  
    var taxaEstorno: Float = 0.0F,  
    var estornaValor: Boolean = false,  
    var isEventActive: Boolean = false,  
    var tipoEvento: TipoEvento,  
    var modalidade: Modalidade,  
    var organizadorEmail: String, // FK para User  
    var ingressosVendidos: Int = 0  
) {  
    override fun toString(): String {  
        val status = if (isEventActive) "[ON]" else "[OFF]"  
        val ocupacao = "$ingressosVendidos/$capacidadeMaxima"  
        val dataStr = "${dataInicio}"  
  
        return "${status.padEnd(6)} | ${nomeEvento.padEnd(18)} | ${dataStr.padEnd(10)} | ${ocupacao.padEnd(10)} | R$ ${precoUnitario.toString().padEnd(6)}"  
    }  
}
```

4.3 Apresentação do Model.kt



```
data class Ingresso(  
    val id: Int,  
    val nomeDoEvento: String,  
    val emailDono: String,  
    val dataDoEvento: LocalDateTime,  
    var status: String = "ATIVO",  
    val valorPago: Float  
) {  
    override fun toString(): String {  
        val idStr = "#${id.toString().padStart(3, '0')}}"  
        return "${idStr.padEnd(6)} | ${nomeDoEvento.padEnd(18)} | ${status.padEnd(10)} | R$ ${valorPago.toString().padEnd(7)}"  
    }  
}
```

5. Operações de repositório



A refatoração do sistema Dendê Eventos centraliza a gestão de dados no arquivo **Repositorio.kt**, adotando os seguintes princípios técnicos:

- **Arquitetura Singleton (object)**
- **Modularização de CRUD**
- **Consultas Especializadas**
- **Ordenação Automática.**

```
import model.*
import kotlinx.datetime.*

object Repositorio {
    // Banco de Dados em Memórias
    private val usuarios = mutableListOf<User>()
    private val eventos = mutableListOf<Evento>()
    private val ingressos = mutableListOf<Ingresso>()

    // --- CRUD USUÁRIOS ---
    fun salvarUsuario(usuario: User) {
        usuarios.add(usuario)
    }

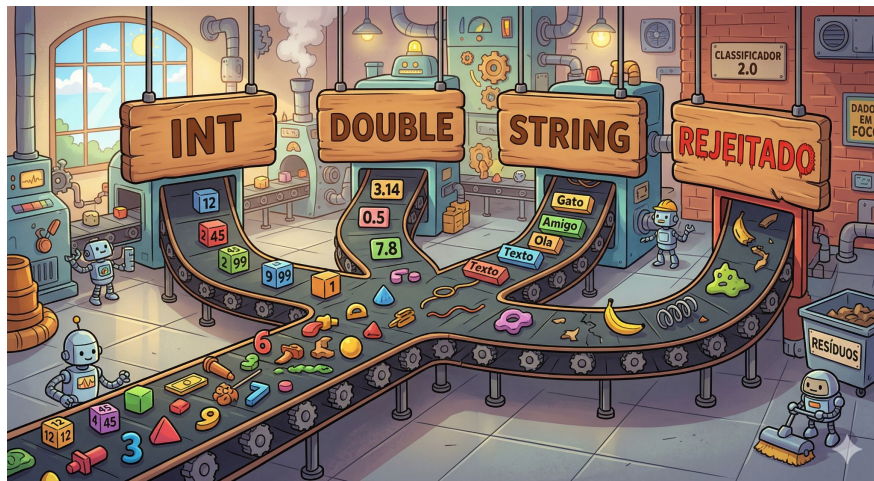
    fun buscarUsuarioPorEmail(email: String): User? {
        return usuarios.find { it.email.equals(email, ignoreCase = true) }
    }
}
```

6. Funções de Componente e Dois Formulários Criado com Elas

Leitura segura:

Implementação de funções que validam os dados imputados pelos usuário, garantindo que eles atendam os critérios estabelecidos:

- ReadString
- ReadDouble
- ReadInt
- ReadGender
- ReadLocalDate
- ReadEmail
- ReadCNPJ
- ReadLocalDateTime
- Tratamento de null



6.1 Funções de Componente e Dois Formulários Criado com Elas

- **Saída padronizada :**

Criação da função `printTable` para exibição das coleções em forma de tabela.



```
// imprime os elementos de uma coleção em formato de tabela
fun printTable(
    //String representando o cabeçalho
    header: String,
    // coleção de objetos que serão usados e impressos na tabela.
    items: List<Any>)
//retorno UNIT implícito
{

    // gera um separador visual com base no comprimento do cabeçalho, garantindo um mínimo de 30 char
    val separator = "-".repeat(header.length.coerceAtLeast(30))

    println(separator)
    //printa o cabeçalho da tabela
    println(header)
    println(separator)

    when {
        // se não tiver nada na lista, um aviso é emitido
        items.isEmpty() -> println("Nenhum registro encontrado.")
        // se tiver dados na lista, passa por cada um deles e imprime
        else -> items.forEach { println(it.toString()) }
    }

    // mostra mais um separador para fechar a tabela
    println(separator)
}
```

7. Resultado da Refatoração do Método main

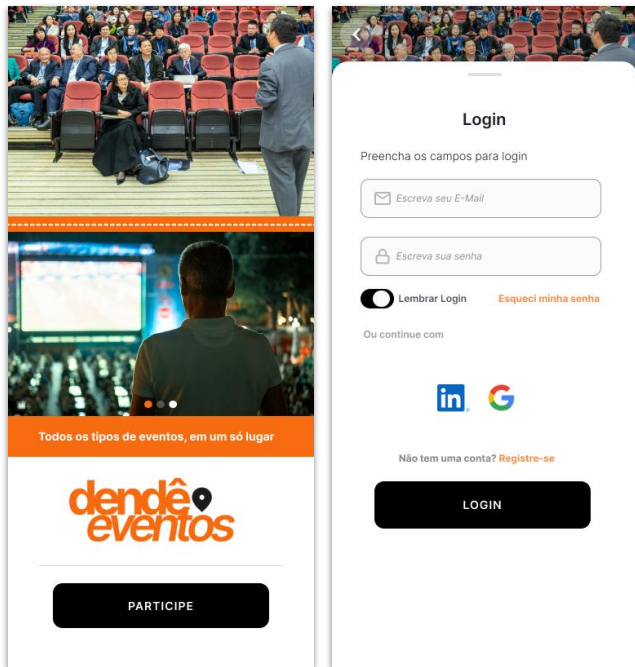
- Fluxo de Controle Simplificado
- Delegação de Responsabilidades
- Integração com Componentes
- Manutenibilidade

```
fun main() {
    var usuarioLogado: User? = null // Começa vazio
    var sistemaAtivo = true
    val data = Clock.System.now().toLocalDateTime(TimeZone.currentSystemDefault())

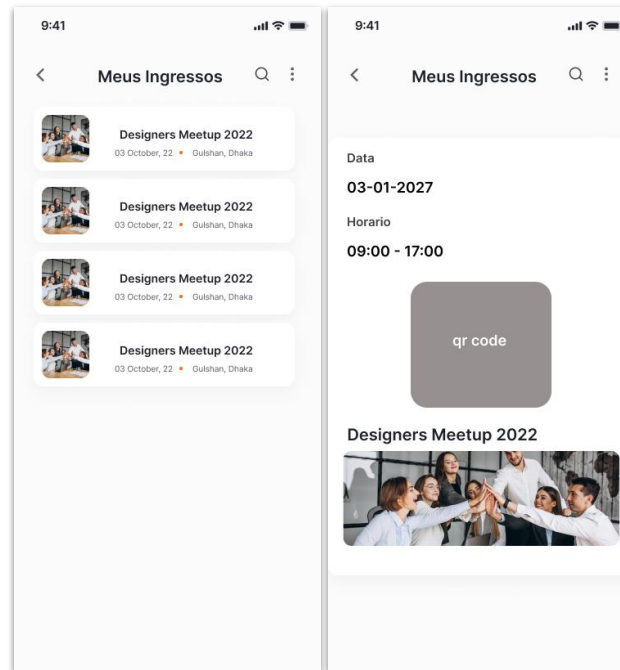
    do {
        println(" - - - Dendê Eventos - - -")
        println("Bem-vindo à plataforma Dendê Eventos! Escolha a opção para prosseguir: ")
        val opcaoIni = readInt(
            message = "1 -> Usuário - CADASTRAR:\n2 -> Organizador - CADASTRAR: ",
            errorMessage = "Erro! Digite apenas 1 ou 2.",
            range = 1..2
        )
    }
```

8. Projeto de Telas

- Telas de Login



- Telas de Listagem de Ingressos



Referências



- JETBRAINS. Kotlin Documentation. Disponível em: <https://kotlinlang.org/docs/home.html>.
- Referência: MARTIN, Robert C. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.